Least-rotation canonicalization at n = 131: exhaustive, zero-gap scan and branch-free, in three limbs

2026-10-14

Least-rotation canonicalization at n = 131

Status: measured. All arms are exact on every word checked. Instruction counts are deterministic (cachegrind). Wall-clock times are exploratory and were taken on an unisolated cloud VM. No speedup is claimed. No curve or ECDLP result is involved: this is word-level canonicalization only. The frozen protocol is `protocol.json`, committed in its own commit `98abad0d` before any metric was collected.

Question

The paper's Section 4 (`papers/frobenius-orbits/sec-canonicalization.tex`) makes normal-basis canonicalization exact and cheaper through the zero-gap fact (Lemma `lem:zerogap`): the least rotation of a nonconstant cyclic word starts at a longest cyclic zero run. It has been implemented and measured for one 64-bit word (variant A, `crypto:src/cryptanalysis/koblitz_fast.rs`, `FrobeniusCanon::least_rotation`) and for two limbs at n = 83 (variant B, `canonical_gap` in `experiments/koblitz-n83-index-design-20260929/packed_index_e1_canonical_20261009/n`
This experiment asks two questions about n = 131, the ECC2K-130 field:

1. Are an exhaustive scan, a variant-B zero-gap scan and a branch-free version exact on a three-limb word?
2. What do they cost in deterministic units: candidate rotations and retired instructions?

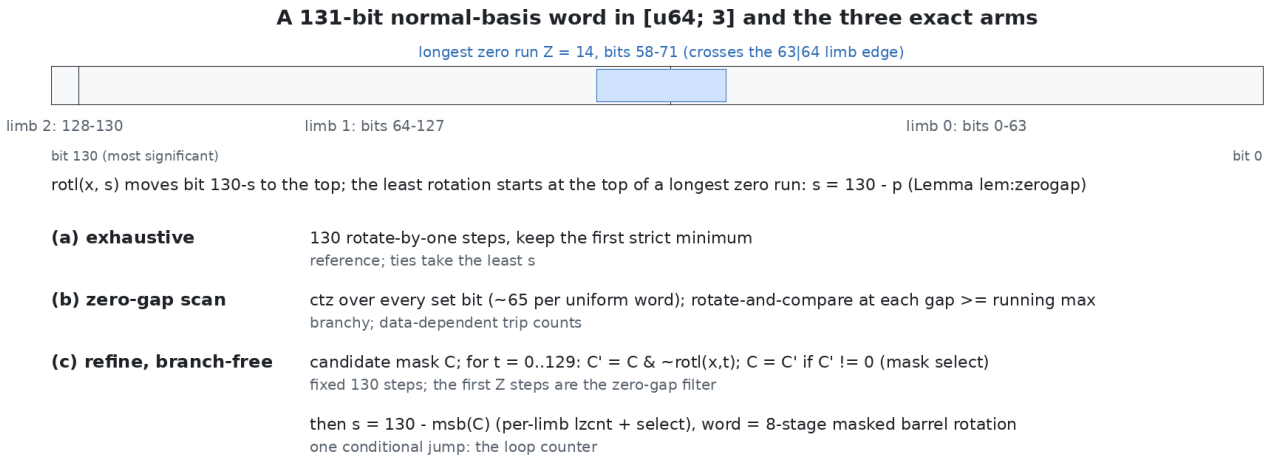


Figure 1: The word layout and the three arms (`figures/layout.svg`).

Representation: `[u64; 3]`, limbs of 64, 64 and 3 bits

Bit `i` is normal-basis coefficient `i`, and limb `l` holds bits `64l` to `64l+63`. The rotation is `rotl(x,s) = ((x<<s) | (x>>(131-s))) & (2^131-1)`, and the canonical form is `(min_s rotl(x,s), least such s)`. This is the same convention as `canonical_gap` at n = 83. I chose three explicit limbs over `u128` plus a top limb for three reasons:

- **Same registers either way.** x86-64 has no 128-bit general-purpose registers, so a `u128` is two `u64` registers anyway. A `u128` hides the cross-limb carries in the compiler's lowering of 128-bit shifts, and a variable `u128` shift lowers to `shld`/`shl` plus a bit-6 test and `cmov`. That would make the branch-free audit depend on the compiler's choices.
- **One generic code path.** The code is generic over the limb width `B`. The same functions run exhaustively at small `n` with 3- to 7-bit limbs, so the cross-limb carries, a partial or empty top limb, and the wrap from bit `n-1` to bit 0 are all exercised on every word.
- **Same layout as the walk.** It is the host layout of `ecc2k130/src/f131.h` (`struct F131 { uint64_t w[3]; }`, 64/64/3), so a canonicalizer built here needs no repacking there.

Arms

arm	what it does
(a) `exhaustive`	130 rotate-by-one steps; keeps the first strict minimum, so ties take the least shift (reference)
(b) `gap_scan`	variant B on three limbs: ctz over the set bits of each limb; at every gap >= the running maximum, rotate and compare; the wrap gap is handled last; constant words return `(x, 0)`
(c) `refine_bf`	branch-free bit-parallel refinement, described below
`exhaustive_bf`	supplementary: (a) with a borrow-chain compare and mask selects (a scalar "variant C")
`lift_scan`	supplementary: a variant-A analogue; branch-free binary lifting of the longest-run mask using constant rotations only (`Z_{2a} = Z_a & rotl(Z_a,a)`, `Z_{l+2^k} = Z_{2^k} & rotl(Z_l,2^k)`), then a data-dependent loop over the `c` run tops

How (c) works. `C` is a mask of candidate positions, and `B_t = rotl(x,t)` holds bit `t` of every candidate's rotation at that candidate's position. Each of the 130 steps computes `C' = C & ~B_t` and keeps `C'` if it is nonzero, else `C`, by mask select. The first `Z` steps are exactly the lemma's zero-run growth. The rest separate the tied candidates, which a fixed-count algorithm must also budget for.

When the loop ends, the code takes `p = msb(C)` with a per-limb `bsr` and a mask select, sets `s = 130 - p`, and forms the word with an 8-stage masked barrel rotation. The constant words (`x` in F_2) keep every candidate, so they get `s = 0` without a special case. The 131st step is skipped because it never splits candidates: every rotation has the same weight.

What "branch-free" means here, and how it was checked

Definition. In the arm's machine code, every conditional jump is a backward loop edge. Its flags come from a counter register that is set by a constant and stepped only by constants inside the loop. The arm makes no `call` and no indirect jump, and its selections use `and`/`andn`/`or` masks. The only memory accesses are the fixed loads of the input and stores of the output.

`asm_check.py` applies this definition mechanically to the built binary with `objdump`; the result is `results/asm_check.json`.

- **`refine_bf`: 1 conditional jump** (`dec r10d; jne`, the 130-step counter). The hot loop is 35 static instructions of shifts and masks with no `cmov`. The `msb` and barrel stages are straight-line code.
- **`exhaustive_bf`: 1 conditional jump** (`cmp r11,0x83; jne`). The compare compiles to `setb`/`sete` arithmetic.
- **Branchy arms, for reference:** `gap_scan` has 57 conditional jumps, `lift_scan` 27 and `exhaustive` 5.
- **Not claimed:** constant time on any particular microarchitecture. No timing-leak test was run.

Finding: source-level masking was not enough. The first build of `exhaustive_bf` wrote its select as `(a & m) | (b & !m)`, and LLVM compiled it to a data-dependent `jne`. The fix is a value barrier: an empty `asm!` block that emits no instruction and hides the mask from the optimizer (`barrier` in `src/lib.rs`). After the fix both branch-free arms pass the check. A branch-free claim needs this object-code check every time the toolchain changes.

Exactness

There were **zero mismatches** in every family (`results/exactness.json`, `results/tests.txt`).

family	words	mismatches
small n: every word, n = 2..20, three small limbs and one 64-bit limb, plus n = 8, 12, 20 with an empty top limb; composite n gives periodic ties; all arms vs exhaustive vs an independent `u64` oracle	5,247,224	0
n = 131 uniform random (seed 1312026101401)	10,000,000	0
x in F_2: 0 and all-ones (the element 1 in a normal basis), each giving `(x, 0)` in every arm	2	0
weight 1, 2, 3 and their complements (weight 128-130); triples with repeats	766,612	0
one zero run of length 1-60 straddling bit 64, bit 128 or the wrap 130 to 0, at every offset; ones background, plus a tied second run, plus 4 random backgrounds	34,020	0
exactly `c` tied longest runs of length `z`, `c` = 2..12, `z` = 1..12, 200 words per cell	25,352	0
near-periodic: period 2-40, 0-2 flips	39,000	0
exhaustive vs a `Vec<bool>` oracle sharing no limb code (half uniform, half structured)	200,000	0
uniqueness of the minimizing shift for nonconstant words (Corollary `cor:unique`)	200,000	0 non-unique

Five tied cells cannot exist, because `c(z+1) > 131`: (11,11), (11,12), (12,10), (12,11) and (12,12). The cell (12,1) reached 152 words within its attempt cap; every other cell has 200.

Test power. Four mutants were each killed by the small-n test: a flipped tie rule, a dropped wrap gap, 129 instead of 130 refinement steps, and a lowest-limb-only `msb`.

Deterministic counts

Source: `results/metrics_uniform.json` (10^7 uniform words, seed 20261014) and `results/metrics_structured.json` (10^6 structured words, seed 20261015). The structured distribution repeats a random pattern with period 2-20 over the 131 bits and flips one bit with probability 1/2. It is a stress test, not a workload model; 26,311 of its words are constant and are skipped.

per word	uniform mean	sd	median	max	structured mean	max
longest zero run `Z`	6.43	1.83	6	26	5.16	130
longest runs `c` (A-type candidate rotations)	**1.436**	0.887	1	20	12.68	65
gap-scan rotate-and-compares	**5.433**	2.485	5	29	18.19	65
refinement steps to one candidate	7.31	1.91	7	31	45.53	130

- **One longest run:** 72.39% of uniform words (7,239,306 of 10^7) and 28.26% of nonconstant structured words.
- **`c` histogram (uniform):** 1: 7,239,306; 2: 1,793,439; 3: 597,349; 4: 220,171; 5: 85,908; >=6: 63,827.

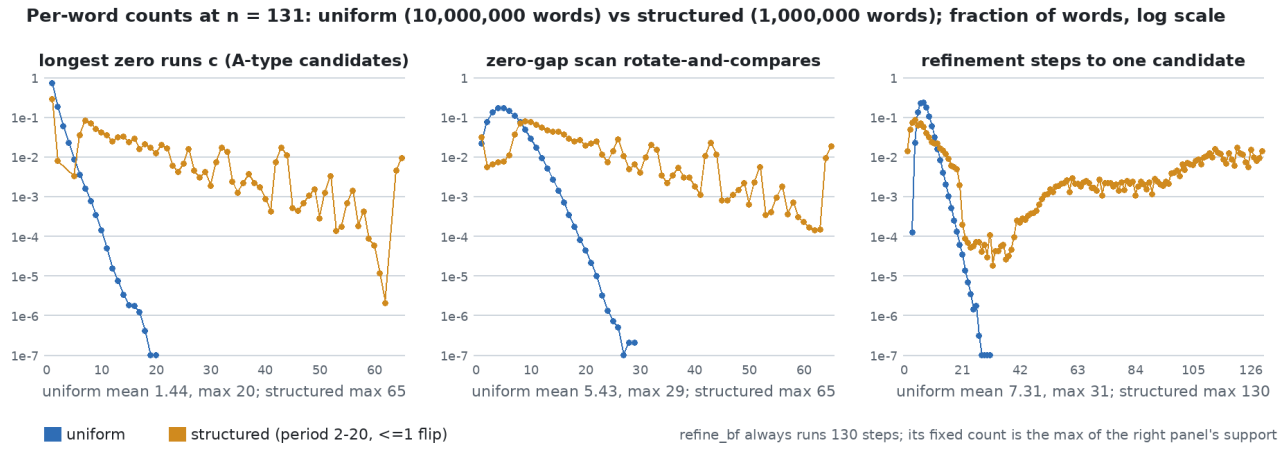


Figure 2: Per-word count distributions (`figures/distributions.svg`).

Why the branch-free arm cannot stop at the filter. On uniform words, one candidate is left after at most 31 steps. Near-periodic words need up to 130 steps, the full `n - 1`. An exact algorithm with a fixed iteration count therefore pays for 130 steps on every word. The zero-gap filter is its first `Z` steps, but it cannot shorten the loop.

Retired instructions

`perf` is not installed on this host (`perf: command not found`), so every count comes from `valgrind --tool=cachegrind --cache-sim=no` (3.22.0).

Method. The figure is each arm symbol's Ir over 200,000 words, divided by 200,000. `asm_check` confirms every arm is a leaf function, and `run.py` refuses to count an arm that contains a call. Each figure includes 6 Ir of call and copy overhead, which is what the `null` arm costs. Source: `results/instructions.json`.

arm	Ir/word, uniform	Ir/word, structured	x exhaustive (uniform)	x gap_scan (uniform)	/ one 131-bit multiply (proxy)
(a) exhaustive	1,849.6	1,855.6	1.00	1.29	21.0
(b) gap_scan	1,434.9	2,738.2	0.78	1.00	16.3

arm	Ir/word, uniform	Ir/word, structured	x exhaustive (uniform)	x gap_scan (uniform)	/ one 131-bit multiply (proxy)
(c) refine_bf	**4,510.0**	**4,510.0**	2.44	**3.14**	51.3
exhaustive_bf	3,664.0	3,664.0	1.98	2.55	41.6
lift_scan	**513.6**	1,478.3	0.28	0.36	5.8
null (overhead)	6.0	6.0			
131-bit polynomial-basis multiply (proxy)	88.0				1

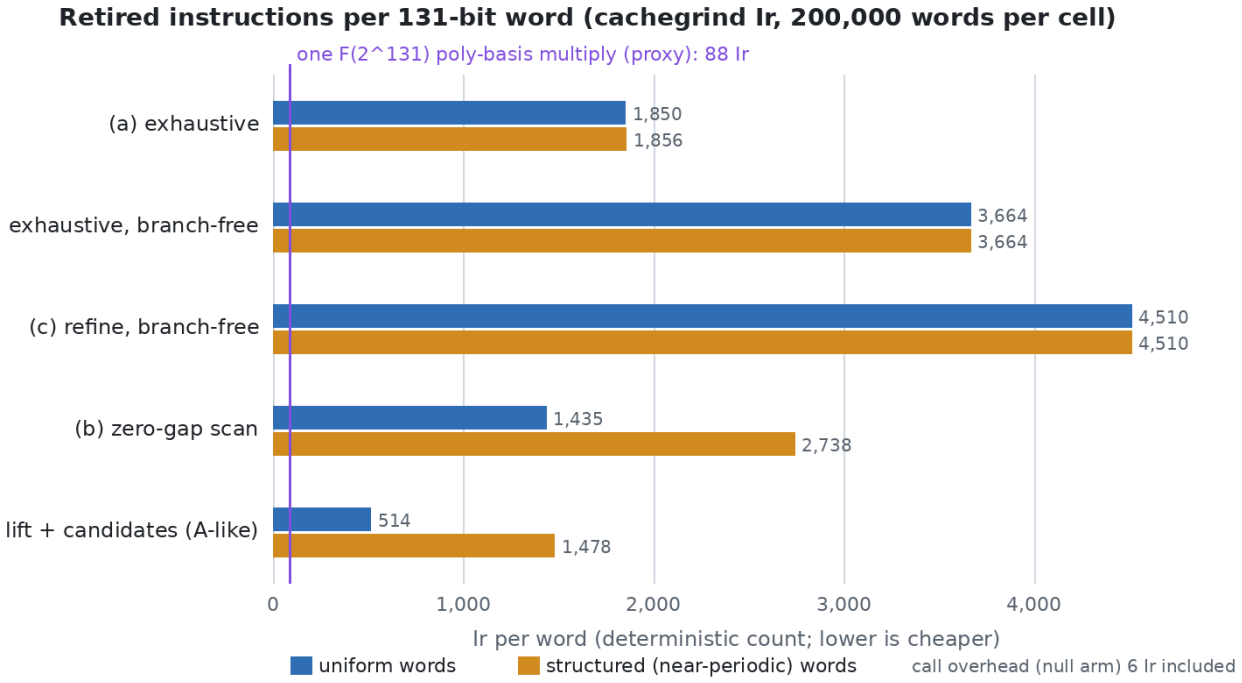


Figure 3: Ir per word by arm and distribution (`figures/ir_per_word.svg`).

Determinism check. The first full run's build and the final build produced identical Ir totals for the four arms whose code did not change (exhaustive, refine_bf, exhaustive_bf, lift_scan) and for `null`, on both distributions. The 10^7-word metrics file is byte-identical between the two runs as well.

Normal-basis multiply. Neither `crypto` nor this crate has a 131-bit normal-basis multiply. In `crypto`, the type-II ONB in `koblitz_rotated_chain.rs` requires m < 128, and `cyclotomic_factor_base.rs` requires n <= 62. So **the cost relative to one normal-basis point operation was not measured.**

The last column is a labelled proxy only. It divides by one 131-bit polynomial-basis product modulo `z^131+z^13+z^2+z+1`: nine PCLMULQDQ products plus a word-level reduction, checked against bitwise schoolbook on 2,000 pairs (`fmul_matches_schoolbook`). I checked that the modulus is irreducible (`x^(2^131) = x` and it has no roots). A point operation in the ECC2K-130 walk adds several multiplies and a basis conversion (`ecc2k130/src/f131.h`), so a point-operation ratio would be several times smaller than this column. It was not measured.

Wall clock (exploratory, unisolated)

These times were taken on an unisolated, shared cloud VM (Intel Xeon @ 2.10 GHz, 4 vCPU) with no isolation receipt. They are not evidence for a speedup under AGENTS.md's CPU tiers. Each figure is the median of 11 passes over 10^6 words; the full rows, with min and max, are in `results/wallclock.json`.

ns/word	exhaustive	gap_scan	refine_bf	exhaustive_bf	lift_scan	null	fmul proxy
uniform	366	420	430	556	62.5	2.0	26.0
structured	340	461	416	584	178	2.3	

Predictions (frozen) and outcomes

	prediction	outcome
P1	every arm exact	held: 0 mismatches
P2	gap-scan rotations per uniform word in [4, 6.5] (4.78 at n = 83)	held: 5.43
P3	fraction with one longest run in [0.65, 0.78] (72.6% at n = 61)	held: 0.7239
P4	the branch-free arms retire identical Ir on both distributions	held exactly: 902,000,000 and 732,800,000 on each
P5	refine_bf costs at least 3x gap_scan's Ir per uniform word	held: 3.14x
P6	uniform refinement depth far below 130; structured depth above 100	held: 31 and 130

Findings, including negative ones

1. **The branch-free arm is the most expensive.** `refine_bf` retires 4,510 Ir per word: 3.14x the zero-gap scan and 2.44x the plain exhaustive scan. It is also 1.23x the branch-free exhaustive chain, the simplest branch-free alternative. Its one advantage is that its cost is independent of the data: on structured words it is only 1.65x the scan's 2,738 Ir. A fixed iteration count has to budget for the worst case, which is 130 steps (P6).
2. **The variant-B scan barely improves on exhaustive at n = 131.** It still cuts rotations from 130 to 5.43 per word. But it also takes a ctz step for every one bit, about 65.5 per uniform word. Its Ir is therefore only 1.29x below the exhaustive scan. On near-periodic words it is 1.48x above it, and in this unisolated run its wall time was above exhaustive on both distributions.
3. **This bears on the paper's cost proposition.** Proposition `prop:cost` counts variant B's ctz scan inside "O(L log n) word operations". The scan visits `t` one bits, and `t` averages n/2 for uniform words. So at n = 131 the scan, not the rotations, sets the cost; that matches the Ir above. I have not edited the paper; this note is for its owner.
4. **The cheapest exact arm is a variant-A analogue.** `lift_scan` uses branch-free lifting (15 constant-rotation stages, no scan over one bits) followed by `c` candidate rotations, 1.44 on average. It needs 514 Ir per uniform word: 3.6x below exhaustive and 2.8x below the zero-gap scan. In exploratory wall time it was also fastest (62.5 ns). It is not branch-free; its loop over candidates runs `c` times.
5. **Compilers can undo source-level masking.** Masked selects are not branch-free until the object code says so: LLVM turned one into a `jne`, and a value barrier fixed it.

Deviations from the protocol

- **Instrument fix and full rerun.** After the first full run, two Ir rows turned out to be instrument artefacts. The `gap_scan` closure had been compiled out of line, and the `fmul` wrapper did not inline its `target_feature` body. I made both inline and added a leaf-function guard to `run.py`, then reran everything. The superseded table and an explanation are kept in `results/superseded_run1/`. The other rows reproduced exactly.
- **Smoke run.** Before the first full run, a `--quick` smoke run checked the runner end to end; its outputs were overwritten.

Graphs checked (research-visuals)

- **Created:** the three new figures in `figures/`; each SVG is generated by `make_figures.py` from `results/*.json`, and the PNG is rasterized from it.
- **No curve graph affected:** no curve, isogeny or volcano graph is involved, and `experiments/volcano-ic/figures/` and `experiments/ic-candidate-catalog/curves.yaml` are unchanged.
- **Paper not updated in this change:** `papers/frobenius-orbits/sec-canonicalization.tex` and its PDF have no n = 131 figure to update. Finding 3 and these numbers are offered to the paper's owner for Section `sec:bench-canon`.

Reproduce

```
export CARGO_TARGET_DIR=...      # optional shared build dir
python3 run.py                   # build (--locked --offline), tests, asm audit, exactness,
                                 # metrics, cachegrind, wall clock -> results/
python3 make_figures.py          # figures/*.svg and *.png
pandoc README.md -o report.pdf --pdf-engine=lualatex
```

The crate has no dependencies and its `Cargo.lock` is committed, so it builds offline. All words are generated in memory from the seeds in `protocol.json`; nothing large is written to disk. `results/build_receipt.json` records the toolchain, the host, and the SHA-256 of every source file and binary.